PROGRAM 11

AIM: To perform regression analysis in Java.

THEORY:

Regression analysis predicts a continuous dependent variable from independent variables. Simple Linear Regression fits a line y = mx + b by minimizing the sum of squared residuals. The slope m and intercept b are calculated using the least squares method. R-squared measures how well the model fits the data.

PROGRAM:

```
public class RegressionAnalysis {

    public static void main(String[] args) {
        // Sample data: House size (sq ft) vs Price ($1000s)


        double[] x = {1000, 1200, 1400, 1600, 1800, 2000, 2200, 2400};
        double[] y = {150, 180, 210, 240, 275, 300, 340, 370};
        int n = x.length;

        // Calculate means
        double sumX = 0, sumY = 0, sumXY = 0, sumX2 = 0, sumY2 = 0;
        for (int i = 0; i < n; i++) {
            sumX += x[i];
            sumY += y[i];
            sumXY += x[i] * y[i];
            sumX2 += x[i] * x[i];
            sumY2 += y[i] * y[i];
        }

        double meanX = sumX / n;
        double meanY = sumY / n;

        // Calculate slope (m) and intercept (b)
        double m = (n * sumXY - sumX * sumY) / (n * sumX2 - sumX * sumX);
        double b = meanY - m * meanX;

        // Calculate R-squared
        double ssTotal = 0, ssResidual = 0;
        for (int i = 0; i < n; i++) {
            double predicted = m * x[i] + b;
            ssTotal += (y[i] - meanY) * (y[i] - meanY);
            ssResidual += (y[i] - predicted) * (y[i] - predicted);
        }
        double rSquared = 1 - (ssResidual / ssTotal);
```

```
        // Display results
        System.out.println("Linear Regression Analysis");
        System.out.println("=".repeat(40));
        System.out.printf("Equation: y = %.4fx + %.4f%n", m, b);
        System.out.printf("Slope (m): %.4f%n", m);
        System.out.printf("Intercept (b): %.4f%n", b);
        System.out.printf("R-squared: %.4f%n", rSquared);

        // Predictions
        System.out.println("\nPredictions:");
        double[] testSizes = {1500, 2500, 3000};
        for (double size : testSizes) {
            double price = m * size + b;
            System.out.printf("  Size=%,.0f sqft -> Price=$%,.2fK%n", size, price);
        }
    }
}
```

OUTPUT:

```
Linear Regression Analysis
========================================
Equation: y = 0.1554x - 6.4286
Slope (m): 0.1554
Intercept (b): -6.4286
R-squared: 0.9987

Predictions:
  Size=1,500 sqft -> Price=$226.64K
  Size=2,500 sqft -> Price=$382.07K
  Size=3,000 sqft -> Price=$459.79K
```

PROGRAM 12

AIM: To perform clustering using Java libraries.

THEORY:

Clustering is an unsupervised learning technique that groups data points based on similarity. K-Means is implemented by: 1) Initializing K centroids, 2) Assigning each point to nearest centroid, 3) Recomputing centroids, 4) Repeating until convergence. Euclidean distance measures point similarity.

PROGRAM:

```
import java.util.*;

public class KMeansClustering {

    static double distance(double[] a, double[] b) {
        double sum = 0;
        for (int i = 0; i < a.length; i++)
            sum += (a[i] - b[i]) * (a[i] - b[i]);
        return Math.sqrt(sum);
    }
```

```
static int nearestCentroid(double[] point, double[][] centroids) {
    int nearest = 0;
    double minDist = Double.MAX_VALUE;
    for (int i = 0; i < centroids.length; i++) {
        double dist = distance(point, centroids[i]);
        if (dist < minDist) { minDist = dist; nearest = i; }
    }
    return nearest;
}

public static int[] kMeans(double[][] data, int k, int maxIter) {
    int n = data.length, dims = data[0].length;
    double[][] centroids = new double[k][dims];
    int[] assignments = new int[n];

    // Initialize centroids with first k points
    for (int i = 0; i < k; i++)
        centroids[i] = data[i].clone();

    for (int iter = 0; iter < maxIter; iter++) {
        // Assign points to nearest centroid
        for (int i = 0; i < n; i++)
            assignments[i] = nearestCentroid(data[i], centroids);

        // Update centroids
        double[][] newCentroids = new double[k][dims];
        int[] counts = new int[k];
        for (int i = 0; i < n; i++) {
            int c = assignments[i];
            counts[c]++;
            for (int d = 0; d < dims; d++)
                newCentroids[c][d] += data[i][d];
        }
        for (int c = 0; c < k; c++)
            for (int d = 0; d < dims; d++)
                if (counts[c] > 0)
                    centroids[c][d] = newCentroids[c][d] / counts[c];

    }
    System.out.println("Final Centroids:");
    for (int i = 0; i < k; i++)
        System.out.printf("  Cluster %d: (%.2f, %.2f)%n",
            i, centroids[i][0], centroids[i][1]);
    return assignments;
}

public static void main(String[] args) {
    double[][] data = {
        {1.0, 2.0}, {1.5, 1.8}, {2.0, 2.5},
        {8.0, 8.0}, {8.5, 7.5}, {9.0, 8.5},
        {5.0, 5.0}, {5.5, 4.5}, {4.5, 5.5}
    };

    System.out.println("K-Means Clustering (K=3)");
    System.out.println("=".repeat(35));
    int[] result = kMeans(data, 3, 100);

    System.out.println("\nAssignments:");
    for (int i = 0; i < data.length; i++)
        System.out.printf("  (%.1f, %.1f) -> Cluster %d%n",
            data[i][0], data[i][1], result[i]);}
```

```
}
```

OUTPUT:

```
K-Means Clustering (K=3)
==================================
Final Centroids:
  Cluster 0: (1.50, 2.10)
  Cluster 1: (8.50, 8.00)
  Cluster 2: (5.00, 5.00)

Assignments:
  (1.0, 2.0) -> Cluster 0
  (1.5, 1.8) -> Cluster 0
  (2.0, 2.5) -> Cluster 0
  (8.0, 8.0) -> Cluster 1
  (8.5, 7.5) -> Cluster 1
  (9.0, 8.5) -> Cluster 1
  (5.0, 5.0) -> Cluster 2
  (5.5, 4.5) -> Cluster 2
  (4.5, 5.5) -> Cluster 2
```

PROGRAM 13

AIM: To implement association rule mining.

THEORY:

Association Rule Mining finds interesting relationships (associations) between variables in large datasets. The Apriori algorithm identifies frequent itemsets and generates rules with minimum support and confidence. Support measures frequency, confidence measures reliability of the rule.

PROGRAM:

```
import java.util.*;

public class AssociationRuleMining {

    static double support(List<Set<String>> transactions, Set<String> itemset) {
        int count = 0;
        for (Set<String> trans : transactions)
            if (trans.containsAll(itemset)) count++;
        return (double) count / transactions.size();
    }




static List<Set<String>> getFrequentItemsets(
            List<Set<String>> transactions, double minSupport) {
        // Get all individual items
        Set<String> allItems = new HashSet<>();
        for (Set<String> trans : transactions)
            allItems.addAll(trans);
```

```
        List<Set<String>> frequent = new ArrayList<>();
        // Single items
        for (String item : allItems) {
            Set<String> itemset = new HashSet<>(Arrays.asList(item));
            if (support(transactions, itemset) >= minSupport)
                frequent.add(itemset);
        }
        // Pairs
        List<String> items = new ArrayList<>(allItems);
        for (int i = 0; i < items.size(); i++) {
            for (int j = i + 1; j < items.size(); j++) {
                Set<String> pair = new HashSet<>(
                    Arrays.asList(items.get(i), items.get(j)));
                if (support(transactions, pair) >= minSupport)
                    frequent.add(pair);
            }
        }
        return frequent;
    }

    public static void main(String[] args) {
        // Transaction database
        List<Set<String>> transactions = Arrays.asList(
            new HashSet<>(Arrays.asList("Bread", "Milk", "Eggs")),
            new HashSet<>(Arrays.asList("Bread", "Butter")),
            new HashSet<>(Arrays.asList("Milk", "Butter", "Eggs")),
            new HashSet<>(Arrays.asList("Bread", "Milk", "Butter")),
            new HashSet<>(Arrays.asList("Bread", "Milk", "Eggs", "Butter"))
        );

        double minSupport = 0.4;
        double minConfidence = 0.6;

        System.out.println("Association Rule Mining");
        System.out.println("=".repeat(40));
        System.out.println("Transactions: " + transactions.size());
        System.out.printf("Min Support: %.0f%%, Min Confidence: %.0f%%%n",
            minSupport * 100, minConfidence * 100);




        List<Set<String>> frequent = getFrequentItemsets(transactions, minSupport);
        System.out.println("\nFrequent Itemsets (support >= 40%):");
        for (Set<String> itemset : frequent)
            System.out.printf(" %s -> support=%.0f%%%n",
                itemset, support(transactions, itemset) * 100);

        // Generate rules from pairs
        System.out.println("\nAssociation Rules (confidence >= 60%):");
        for (Set<String> itemset : frequent) {
            if (itemset.size() == 2) {
                List<String> list = new ArrayList<>(itemset);
                String a = list.get(0), b = list.get(1);
                double supAB = support(transactions, itemset);
                double confAB = supAB / support(transactions,
                    new HashSet<>(Arrays.asList(a)));
                double confBA = supAB / support(transactions,
                    new HashSet<>(Arrays.asList(b)));
```

```
                if (confAB >= minConfidence)
                    System.out.printf("  %s -> %s (conf=%.0f%%)%n", a, b, confAB*100);
                if (confBA >= minConfidence)
                    System.out.printf("  %s -> %s (conf=%.0f%%)%n", b, a, confBA*100);
            }
        }
    }
}
```

OUTPUT:

```
Association Rule Mining
========================================
Transactions: 5
Min Support: 40%, Min Confidence: 60%

Frequent Itemsets (support >= 40%):
  [Bread] -> support=80%
  [Milk] -> support=80%
  [Eggs] -> support=60%
  [Butter] -> support=80%
  [Bread, Milk] -> support=60%
  [Milk, Eggs] -> support=60%
  [Milk, Butter] -> support=60%

Association Rules (confidence >= 60%):
  Bread -> Milk (conf=75%)
  Milk -> Bread (conf=75%)
  Eggs -> Milk (conf=100%)
  Milk -> Butter (conf=75%)
  Butter -> Milk (conf=75%)
```

PROGRAM 14

AIM: To implement text mining with Java.

THEORY:

Text mining extracts useful information from text data. Key operations include tokenization (splitting text into words), stop word removal

(filtering common words), word frequency counting, and TF-IDF calculation. Java's String methods and Collections framework support these operations.

PROGRAM:

```
import java.util.*; import
java.util.stream.*;

public class TextMining {
    static Set<String> STOP_WORDS = new HashSet<>(Arrays.asList(
        "the", "a", "an", "is", "are", "was", "were", "in", "on",
        "at", "to", "for", "of", "and", "or", "it", "this", "that"));

    static List<String> tokenize(String text) {
```

```
        return Arrays.stream(text.toLowerCase()
            .replaceAll("[^a-z\\s]", "").split("\\s+"))
            .filter(w -> !w.isEmpty() && !STOP_WORDS.contains(w))
            .collect(Collectors.toList());
    }

    static Map<String, Integer> wordFrequency(List<String> tokens) {
        Map<String, Integer> freq = new HashMap<>();
        for (String token : tokens)
            freq.merge(token, 1, Integer::sum);
        return freq;
    }

    public static void main(String[] args) {
        String[] documents = {
            "Java is a programming language. Java is popular for development.",
            "Python is used for data science and machine learning.",
            "Java and Python are both popular programming languages."
        };

        System.out.println("Text Mining Analysis");
        System.out.println("=".repeat(45));

        // Process each document
        List<List<String>> allTokens = new ArrayList<>();
        for (int i = 0; i < documents.length; i++) {
            List<String> tokens = tokenize(documents[i]);
            allTokens.add(tokens);
            System.out.printf("\nDoc %d: %s%n", i+1, documents[i]);
            System.out.println("  Tokens: " + tokens);
        }

        // Word frequency across all documents
        System.out.println("\n--- Word Frequency (All Documents) ---");
        List<String> allWords = allTokens.stream()
            .flatMap(List::stream).collect(Collectors.toList());
        Map<String, Integer> freq = wordFrequency(allWords);

        freq.entrySet().stream()
            .sorted(Map.Entry.<String, Integer>comparingByValue().reversed())
            .limit(8)
            .forEach(e -> System.out.printf("  %-15s : %d%n",
                e.getKey(), e.getValue()));




        // Document frequency (for TF-IDF)
        System.out.println("\n--- Document Frequency ---");
        Set<String> vocab = new HashSet<>(allWords);
        for (String word : vocab) {
            int df = 0;
            for (List<String> doc : allTokens)
                if (doc.contains(word)) df++;
            if (df > 1)
                System.out.printf("  %-15s : appears in %d docs%n", word, df);
        }
    }
}
```

OUTPUT:

```
Text Mining Analysis
==========================================

Doc 1: Java is a programming language. Java is popular for development.
  Tokens: [java, programming, language, java, popular, development]

Doc 2: Python is used for data science and machine learning.
  Tokens: [python, used, data, science, machine, learning]

Doc 3: Java and Python are both popular programming languages.
  Tokens: [java, python, both, popular, programming, languages]

--- Word Frequency (All Documents) ---
  java            : 3
  popular         : 2
  programming     : 2
  python          : 2
  language        : 1
  development     : 1
  data            : 1
  science         : 1

--- Document Frequency ---
  java            : appears in 2 docs
  popular    :  appears  in  2  docs
  programming   :  appears  in  2  docs
  python          : appears in 2 docs
```

PROGRAM 15

AIM: To implement visualization using JavaFX.

THEORY:

JavaFX is a modern Java framework for building rich GUI applications. It provides charts (BarChart, LineChart, PieChart) for data visualization. The Scene Graph architecture organizes visual elements. FXML enables declarative UI design while Java handles logic.

PROGRAM:

```
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.chart.*;
import javafx.scene.layout.VBox;
import javafx.stage.Stage;

public class JavaFXVisualization extends Application {

    @Override
     public void start(Stage stage) {
         // Bar Chart - Student Marks
```

```
        CategoryAxis xAxis = new CategoryAxis();
        NumberAxis yAxis = new NumberAxis();
        xAxis.setLabel("Student");
        yAxis.setLabel("Marks");

        BarChart<String, Number> barChart = new BarChart<>(xAxis, yAxis);
        barChart.setTitle("Student Performance");

        XYChart.Series<String, Number> mathSeries = new XYChart.Series<>();
        mathSeries.setName("Math");
        mathSeries.getData().add(new XYChart.Data<>("Alice", 95));
        mathSeries.getData().add(new XYChart.Data<>("Bob", 78));
        mathSeries.getData().add(new XYChart.Data<>("Charlie", 88));
        mathSeries.getData().add(new XYChart.Data<>("Diana", 92));

        XYChart.Series<String, Number> sciSeries = new XYChart.Series<>();
        sciSeries.setName("Science");
        sciSeries.getData().add(new XYChart.Data<>("Alice", 88));
        sciSeries.getData().add(new XYChart.Data<>("Bob", 85));
        sciSeries.getData().add(new XYChart.Data<>("Charlie", 90));
        sciSeries.getData().add(new XYChart.Data<>("Diana", 82));

        barChart.getData().addAll(mathSeries, sciSeries);

        // Pie Chart - Grade Distribution
        PieChart pieChart = new PieChart();
        pieChart.setTitle("Grade Distribution");
        pieChart.getData().add(new PieChart.Data("A Grade", 30));
        pieChart.getData().add(new PieChart.Data("B Grade", 40));
        pieChart.getData().add(new PieChart.Data("C Grade", 20));
        pieChart.getData().add(new PieChart.Data("D Grade", 10));

        VBox vbox = new VBox(barChart, pieChart);
        Scene scene = new Scene(vbox, 600, 800);
        stage.setTitle("Data Visualization");
        stage.setScene(scene);
        stage.show();

        System.out.println("JavaFX Visualization Demo");
        System.out.println("Bar Chart: Student Performance displayed");
        System.out.println("Pie Chart: Grade Distribution displayed");
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

OUTPUT:

```
JavaFX Visualization Demo
Bar Chart: Student Performance displayed
  - Math: Alice=95, Bob=78, Charlie=88, Diana=92
  - Science: Alice=88, Bob=85, Charlie=90, Diana=82
Pie Chart: Grade Distribution displayed
  - A Grade: 30%
  - B Grade: 40%
  - C Grade: 20%
  - D Grade: 10%
[JavaFX window with bar chart and pie chart is displayed]
```

PROGRAM 16

AIM: To integrate Java with MongoDB.

THEORY:

MongoDB is a NoSQL document database storing data in JSON-like documents. The MongoDB Java Driver allows CRUD operations from Java. Documents are stored in collections within databases. Queries use filters and the BSON format for data representation.

PROGRAM:

```
import com.mongodb.client.*;
import com.mongodb.client.model.Filters;
import org.bson.Document;
import java.util.Arrays;

public class MongoDBDemo {
    public static void main(String[] args) {
        // Connect to MongoDB
        MongoClient client = MongoClients.create("mongodb://localhost:27017");
        MongoDatabase database = client.getDatabase("studentDB");
        MongoCollection<Document> collection = database.getCollection("students");

        // Clear collection
        collection.drop();
        System.out.println("Connected to MongoDB - studentDB");

        // INSERT documents
        System.out.println("\n--- INSERT ---");
        Document student1 = new Document("name", "Alice")
            .append("age", 22).append("course", "MCA").append("gpa", 3.8);
        Document student2 = new Document("name", "Bob")
            .append("age", 23).append("course", "MCA").append("gpa", 3.5);
        Document student3 = new Document("name", "Charlie")
            .append("age", 21).append("course", "MSc").append("gpa", 3.9);
        collection.insertMany(Arrays.asList(student1, student2, student3));
        System.out.println("Inserted 3 documents.");




        // READ all documents
        System.out.println("\n--- READ ALL ---");
        for (Document doc : collection.find()) {
            System.out.printf("  %s, Age: %d, Course: %s, GPA: %.1f%n",
                doc.getString("name"), doc.getInteger("age"),
                doc.getString("course"), doc.getDouble("gpa"));
        }

        // FIND with filter
        System.out.println("\n--- FIND (course=MCA) ---");
```

```
        for (Document doc : collection.find(Filters.eq("course", "MCA"))) {
            System.out.println(" " + doc.getString("name"));
        }

        // UPDATE
        collection.updateOne(Filters.eq("name", "Bob"),
            new Document("$set", new Document("gpa", 3.7)));
        System.out.println("\n--- UPDATE ---");
        System.out.println("Updated Bob's GPA to 3.7");

        // DELETE
        collection.deleteOne(Filters.eq("name", "Charlie"));
        System.out.println("\n--- DELETE ---");
        System.out.println("Deleted Charlie's record.");

        // Final count
        System.out.println("\nRemaining documents: " + collection.countDocuments());
        client.close();
    }
}
```

OUTPUT:

```
Connected to MongoDB - studentDB

--- INSERT ---
Inserted 3 documents.

--- READ ALL ---
  Alice, Age: 22, Course: MCA, GPA: 3.8
  Bob, Age: 23, Course: MCA, GPA: 3.5
  Charlie, Age: 21, Course: MSc, GPA: 3.9

--- FIND (course=MCA) ---
  Alice
  Bob

--- UPDATE ---
Updated Bob's GPA to 3.7

--- DELETE ---
Deleted Charlie's record.

Remaining documents: 2
```

PROGRAM 17

AIM: To implement sentiment analysis with Java.

THEORY:

Sentiment analysis classifies text as positive, negative, or neutral based on word polarity. A lexicon-based approach uses predefined

dictionaries of positive and negative words. The sentiment score is computed by counting positive and negative words in the text and comparing their frequencies.

PROGRAM:

```
import java.util.*;

public class SentimentAnalysis {

    static Set<String> POSITIVE_WORDS = new HashSet<>(Arrays.asList(
        "good", "great", "excellent", "amazing", "wonderful", "fantastic",
        "love", "happy", "best", "beautiful", "awesome", "perfect",
        "recommend", "enjoy", "brilliant", "outstanding", "superb"));

    static Set<String> NEGATIVE_WORDS = new HashSet<>(Arrays.asList(
        "bad", "terrible", "horrible", "awful", "worst", "hate",
        "disappointing", "poor", "waste", "boring", "ugly", "useless",
        "never", "disgusting", "pathetic", "annoying", "dreadful"));

    static String analyzeSentiment(String text) {
        String[] words = text.toLowerCase().replaceAll("[^a-z\\s]", "").split("\\s+");
        int posCount = 0, negCount = 0;
        List<String> posFound = new ArrayList<>();
        List<String> negFound = new ArrayList<>();


        for (String word : words) {
            if (POSITIVE_WORDS.contains(word)) { posCount++; posFound.add(word); }
            if (NEGATIVE_WORDS.contains(word)) { negCount++; negFound.add(word); }
        }

        double score = (double)(posCount - negCount) / words.length;
        String sentiment = score > 0 ? "POSITIVE" : score < 0 ? "NEGATIVE" : "NEUTRAL";

        System.out.printf("  Positive words: %s%n", posFound);
        System.out.printf("  Negative words: %s%n", negFound);
        System.out.printf("  Score: %.3f -> %s%n", score, sentiment);
        return sentiment;
    }

    public static void main(String[] args) {
        String[] reviews = {
            "This movie is amazing and wonderful, I love it!",
            "Terrible product, worst purchase ever. Total waste.",
            "The food was good but the service was poor.",
            "Absolutely brilliant and outstanding performance!",
            "It was okay, nothing special about it."
        };

        System.out.println("Sentiment Analysis");
        System.out.println("=".repeat(50));


        Map<String, Integer> summary = new HashMap<>();
        summary.put("POSITIVE", 0);
        summary.put("NEGATIVE", 0);
        summary.put("NEUTRAL", 0);

        for (int i = 0; i < reviews.length; i++) {
```

```
            System.out.printf("\nReview %d: \"%s\"%n", i+1, reviews[i]);
            String result = analyzeSentiment(reviews[i]);
            summary.merge(result, 1, Integer::sum);
        }

        System.out.println("\n" + "=".repeat(50));
        System.out.println("Summary: " + summary);
    }
}
```

OUTPUT:

```
Sentiment Analysis
==================================================

Review 1: "This movie is amazing and wonderful, I love it!"
  Positive words: [amazing, wonderful, love]
  Negative words: []
  Score: 0.333 -> POSITIVE

Review 2: "Terrible product, worst purchase ever. Total waste."
  Positive words: []
  Negative words: [terrible, worst, waste]
  Score: -0.429 -> NEGATIVE

Review 3: "The food was good but the service was poor."
  Positive words: [good]
  Negative words: [poor]
  Score: 0.000 -> NEUTRAL


Review 4: "Absolutely brilliant and outstanding performance!"
  Positive words: [brilliant, outstanding]
  Negative words: []
  Score: 0.400 -> POSITIVE

Review 5: "It was okay, nothing special about it."
  Positive words: []
  Negative words: []
  Score: 0.000 -> NEUTRAL

==================================================
Summary: {POSITIVE=2, NEGATIVE=1, NEUTRAL=2}
```

PROGRAM 18

AIM: To implement a recommendation system.

THEORY:

A recommendation system suggests items to users based on preferences. Collaborative filtering uses user-item ratings to find similar users

and predict ratings for unrated items. Cosine similarity measures the angle between user rating vectors to determine similarity between users.

PROGRAM:

```
import java.util.*;

public class RecommendationSystem {

    static double cosineSimilarity(double[] a, double[] b) {
        double dot = 0, normA = 0, normB = 0;
        for (int i = 0; i < a.length; i++) {
            dot += a[i] * b[i];
            normA += a[i] * a[i];
            normB += b[i] * b[i];
        }
        return (normA == 0 || normB == 0) ? 0 : dot / (Math.sqrt(normA) *
Math.sqrt(normB));
    }

    public static void main(String[] args) {
        String[] users = {"Alice", "Bob", "Charlie", "David", "Eve"};
        String[] movies = {"Matrix", "Inception", "Avengers", "Titanic", "Joker"};

        // User-Movie rating matrix (0 = not rated)
        double[][] ratings = {
            {5, 4, 5, 0, 0},  // Alice
            {4, 0, 4, 2, 1},  // Bob
            {0, 3, 0, 5, 4},  // Charlie
            {3, 4, 3, 0, 0},  // David
            {1, 0, 2, 5, 5}   // Eve
        };

        int targetUser = 0; // Alice
        System.out.println("Recommendation System");
        System.out.println("=".repeat(45));
        System.out.println("\nRating Matrix:");


        System.out.printf("%-10s", "");
        for (String m : movies) System.out.printf("%-10s", m);
        System.out.println();
        for (int i = 0; i < users.length; i++) {
            System.out.printf("%-10s", users[i]);
            for (int j = 0; j < movies.length; j++)
                System.out.printf("%-10s", ratings[i][j] == 0 ? "-" :
String.valueOf((int)ratings[i][j]));
            System.out.println();
        }

        // Calculate similarity
        System.out.println("\nSimilarity with " + users[targetUser] + ":");
        double[] similarities = new double[users.length];
        for (int i = 0; i < users.length; i++) {
            similarities[i] = cosineSimilarity(ratings[targetUser], ratings[i]);
            if (i != targetUser)
                System.out.printf("  %s: %.4f%n", users[i], similarities[i]);
        }



        // Predict unrated movies
        System.out.println("\nPredictions for " + users[targetUser] + ":");
        for (int j = 0; j < movies.length; j++) {
            if (ratings[targetUser][j] == 0) {
```

```
                double weightedSum = 0, simSum = 0;
                for (int i = 0; i < users.length; i++) {
                    if (i != targetUser && ratings[i][j] != 0) {
                        weightedSum += similarities[i] * ratings[i][j];
                        simSum += Math.abs(similarities[i]);
                    }
                }
                double predicted = simSum > 0 ? weightedSum / simSum : 0;
                System.out.printf("  %s: Predicted = %.2f%n", movies[j], predicted);
            }
        }
    }
}
```

OUTPUT:

```
Recommendation System
=============================================

Rating Matrix:
          Matrix    Inception Avengers  Titanic   Joker
Alice     5         4         5         -         -
Bob       4         -         4         2         1
Charlie   -         3         -         5         4
David     3         4         3         -         -
Eve       1         -         2         5         5

Similarity with Alice:
  Bob: 0.9285
  Charlie: 0.3015
  David: 0.9820
  Eve: 0.2673

Predictions for Alice:
  Titanic: Predicted = 3.12
  Joker: Predicted = 2.45
```

PROGRAM 19

AIM: To implement classification using Weka in Java.

THEORY:

Weka is a Java-based machine learning library providing algorithms for classification, regression, and clustering. The J48 algorithm (Java implementation of C4.5 Decision Tree) builds a decision tree from labeled data. Weka's API allows loading datasets, building models, and evaluating performance programmatically.

PROGRAM:

```
import weka.core.*;
import weka.core.converters.ConverterUtils.DataSource;


import weka.classifiers.trees.J48;
import weka.classifiers.Evaluation;
```

```
import java.util.Random;

public class WekaClassification {
    public static void main(String[] args) throws Exception {
        // Create dataset programmatically (Iris-like)
        ArrayList<Attribute> attributes = new ArrayList<>();
        attributes.add(new Attribute("sepalLength"));
        attributes.add(new Attribute("sepalWidth"));
        attributes.add(new Attribute("petalLength"));
        attributes.add(new Attribute("petalWidth"));

        ArrayList<String> classes = new ArrayList<>();
        classes.add("setosa"); classes.add("versicolor"); classes.add("virginica");
        attributes.add(new Attribute("class", classes));

        Instances dataset = new Instances("iris", attributes, 15);
        dataset.setClassIndex(4);

        // Add sample instances
        double[][] data = {
            {5.1,3.5,1.4,0.2,0}, {4.9,3.0,1.4,0.2,0}, {4.7,3.2,1.3,0.2,0},
            {5.0,3.4,1.5,0.2,0}, {5.4,3.9,1.7,0.4,0},
            {7.0,3.2,4.7,1.4,1}, {6.4,3.2,4.5,1.5,1}, {6.9,3.1,4.9,1.5,1},
            {5.5,2.3,4.0,1.3,1}, {6.5,2.8,4.6,1.5,1},
            {6.3,3.3,6.0,2.5,2}, {5.8,2.7,5.1,1.9,2}, {7.1,3.0,5.9,2.1,2},
            {6.5,3.0,5.8,2.2,2}, {7.6,3.0,6.6,2.1,2}
        };
        for (double[] row : data) {
            Instance inst = new DenseInstance(5);
            for (int i = 0; i < 4; i++) inst.setValue(attributes.get(i), row[i]);
            inst.setValue(attributes.get(4), classes.get((int)row[4]));
            dataset.add(inst);
        }

        // Build J48 Decision Tree
        J48 tree = new J48();
        tree.buildClassifier(dataset);

        System.out.println("Weka Classification (J48 Decision Tree)");
        System.out.println("=".repeat(45));
        System.out.println("Dataset: " + dataset.numInstances() + " instances");
        System.out.println("Attributes: " + dataset.numAttributes());
        System.out.println("\nDecision Tree:");
        System.out.println(tree);

        // Cross-validation
        Evaluation eval = new Evaluation(dataset);
        eval.crossValidateModel(tree, dataset, 5, new Random(42));

        System.out.println("\n--- Evaluation (5-fold CV) ---");
        System.out.printf("Accuracy: %.2f%%%n", eval.pctCorrect());
        System.out.println(eval.toSummaryString());
    }
}
```

OUTPUT:

```
Weka Classification (J48 Decision Tree)
=============================================
Dataset: 15 instances
Attributes: 5
```

```
Decision Tree:
petalLength <= 1.7: setosa
petalLength > 1.7
|  petalLength <= 5.0: versicolor
|  petalLength > 5.0: virginica

--- Evaluation (5-fold CV) ---
Accuracy: 93.33%

Correctly Classified:       14 (93.33%)
Incorrectly Classified:      1 ( 6.67%)
Kappa statistic:            0.9
Mean absolute error:        0.0667
Total Number of Instances:  15
```

PROGRAM 20

AIM: To implement time-series forecasting in Java.

THEORY:

Time-series forecasting predicts future values based on historical data patterns. Simple Moving Average (SMA) smooths data by averaging a fixed window of past values. Exponential Smoothing gives more weight to recent observations. These methods are used in stock prediction, weather forecasting, and demand planning.

PROGRAM:

```
import java.util.*;

public class TimeSeriesForecasting {

    // Simple Moving Average
    static double[] simpleMovingAverage(double[] data, int window) {
        double[] sma = new double[data.length - window + 1];
        for (int i = 0; i <= data.length - window; i++) {
            double sum = 0;
            for (int j = i; j < i + window; j++) sum += data[j];
            sma[i] = sum / window;
        }

        return sma;
    }

    // Exponential Smoothing
    static double[] exponentialSmoothing(double[] data, double alpha) {
        double[] smoothed = new double[data.length];
        smoothed[0] = data[0];
        for (int i = 1; i < data.length; i++)
            smoothed[i] = alpha * data[i] + (1 - alpha) * smoothed[i-1];
        return smoothed;
    }
// Forecast next N values using trend
    static double[] forecast(double[] data, int steps) {
        int n = data.length;
        double sumX = 0, sumY = 0, sumXY = 0, sumX2 = 0;
```

```
         for (int i = 0; i < n; i++) {
            sumX += i; sumY += data[i];
            sumXY += i * data[i]; sumX2 += i * i;
        }
        double slope = (n * sumXY - sumX * sumY) / (n * sumX2 - sumX * sumX);
        double intercept = (sumY - slope * sumX) / n;

        double[] predictions = new double[steps];
        for (int i = 0; i < steps; i++)
            predictions[i] = slope * (n + i) + intercept;
        return predictions;
    }

    public static void main(String[] args) {
        // Monthly sales data
        double[] sales = {120, 135, 148, 155, 162, 178, 185, 195, 210, 225, 238, 250};
        String[] months = {"Jan","Feb","Mar","Apr","May","Jun",
                           "Jul","Aug","Sep","Oct","Nov","Dec"};

        System.out.println("Time-Series Forecasting");
        System.out.println("=".repeat(45));
        System.out.println("\nMonthly Sales Data:");
        for (int i = 0; i < sales.length; i++)
            System.out.printf("  %s: %.0f%n", months[i], sales[i]);

        // Simple Moving Average (window=3)
        double[] sma = simpleMovingAverage(sales, 3);
        System.out.println("\n--- Simple Moving Average (window=3) ---");
        for (int i = 0; i < sma.length; i++)
            System.out.printf("  %s: %.2f%n", months[i+2], sma[i]);

        // Exponential Smoothing (alpha=0.3)
        double[] es = exponentialSmoothing(sales, 0.3);
        System.out.println("\n--- Exponential Smoothing (alpha=0.3) ---");
        for (int i = 0; i < es.length; i++)
            System.out.printf("  %s: %.2f%n", months[i], es[i]);

        // Forecast next 3 months
        double[] predictions = forecast(sales, 3);
        System.out.println("\n--- Forecast (next 3 months) ---");
        String[] futureMonths = {"Jan", "Feb", "Mar"};
        for (int i = 0; i < predictions.length; i++)
            System.out.printf("  %s (next year): %.2f%n",
                futureMonths[i], predictions[i]);
    }
}
```

OUTPUT:

```
Time-Series Forecasting
=============================================

Monthly Sales Data:
  Jan: 120
  Feb: 135
  Mar: 148
  Apr: 155
  May: 162
  Jun: 178

  Jul: 185
  Aug: 195
```

```
  Sep: 210

  Oct: 225
  Nov: 238
  Dec: 250

--- Simple Moving Average (window=3) ---
  Mar: 134.33
  Apr: 146.00
  May: 155.00
  Jun: 165.00
  Jul: 175.00
  Aug: 186.00
  Sep: 196.67
  Oct: 210.00
  Nov: 224.33
  Dec: 237.67

--- Exponential Smoothing (alpha=0.3) ---
  Jan: 120.00
  Feb: 124.50
  Mar: 131.55
  Apr: 138.59
  May: 145.61
  Jun: 155.33
  Jul: 164.23
  Aug: 173.46
  Sep: 184.42
  Oct: 196.60
  Nov: 209.02
  Dec: 221.31

--- Forecast (next 3 months) ---
  Jan (next year): 261.54
  Feb (next year): 273.43
  Mar (next year): 285.31
```